IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute Of Arts Science Commerce & Management

PROJECT – HIGH LEVEL DESIGN

ON

MULTI-CLOUD HYBRID STRATEGY FOR HEALTHCARE SYSTEM

Student Name: Muhammad Razeen TC

Industry Mentor: _____ Guided By: _____

1. Introduction

1.1 Scope of the Document

This document provides the **High-Level Design (HLD)** for the project titled
"Multi-Cloud Hybrid Backup Strategy for Healthcare System."

It includes:

- System architecture
- Backup and restore mechanisms
- Redundancy and failover strategies
- Data handling and workflows

1.2 Intended Audience

- Students (BCA/BSc)
- Developers
- Industry Mentors / Evaluators
- Cloud & DevOps Engineers

1.3 System Overview

The system is a **Django-based Healthcare Management Application** integrated with a **Multi-Source Backup System**.

Features:

- Doctor & patient management
- Appointment booking
- Prescription upload
- Automatic backup trigger
- Local + Cloud backup
- Failover restore

---

2. System Design

2.1 Application Design

Layered Architecture:

- **Presentation Layer** → UI (HTML templates)
- **Application Layer** → Django logic
- **Backup Service Layer** → Python scripts
- **Storage Layer** → Local + Cloud

2.2 Process Flow

Backup:

Doctor → Upload Prescription → Signal → Backup → Local + Cloud

Restore:

Local Backup → If Fail → Cloud Backup → Restore

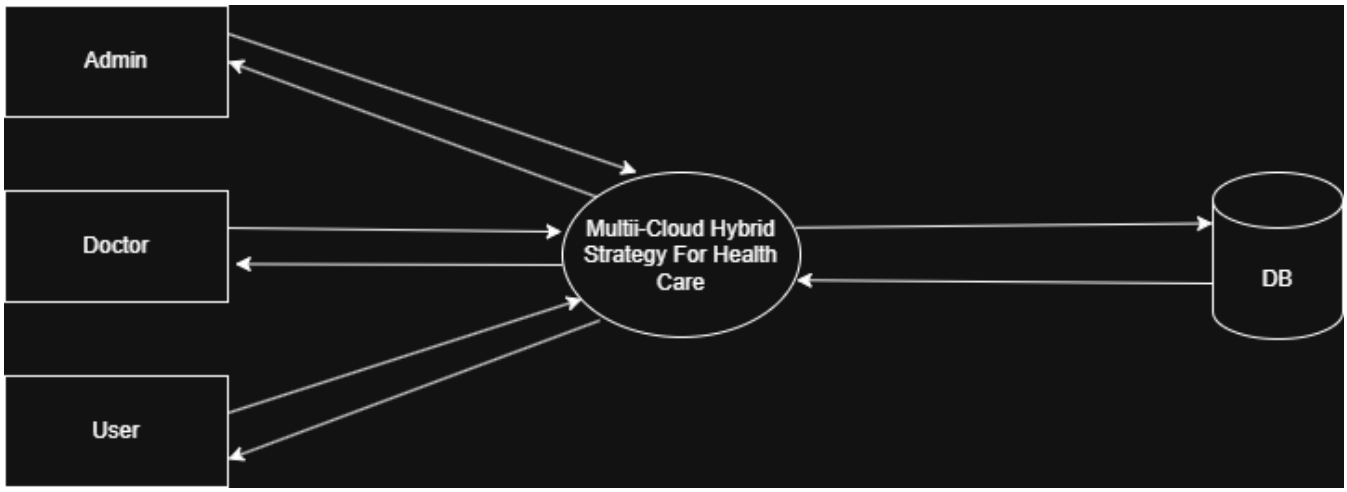
2.3 Information Flow

User → Django App → Database
→ Trigger Signal → Backup Module
→ Local Storage + Cloud Storage

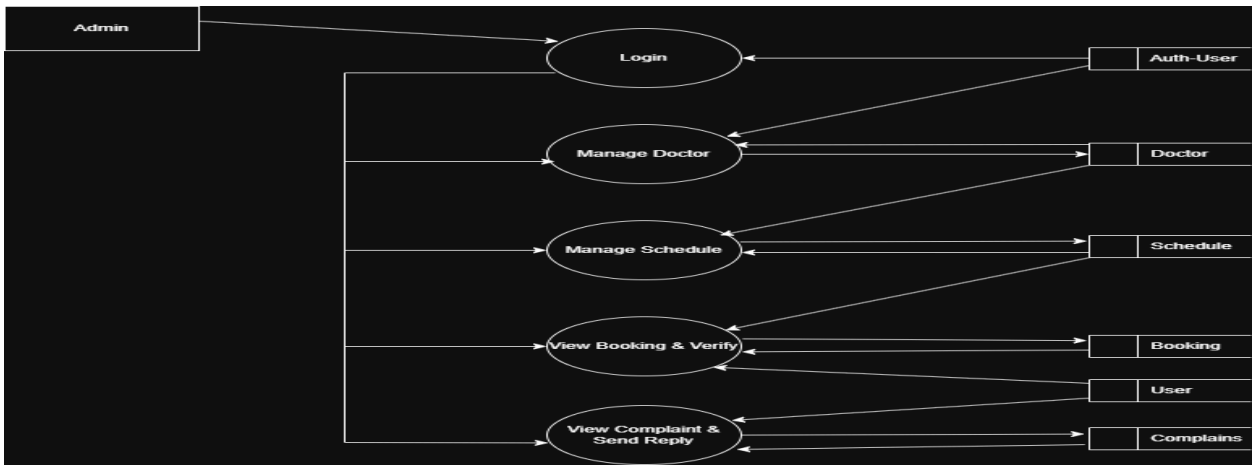
DATA FLOW

- Level 0
- Level 1
- Level 2
- Level 3

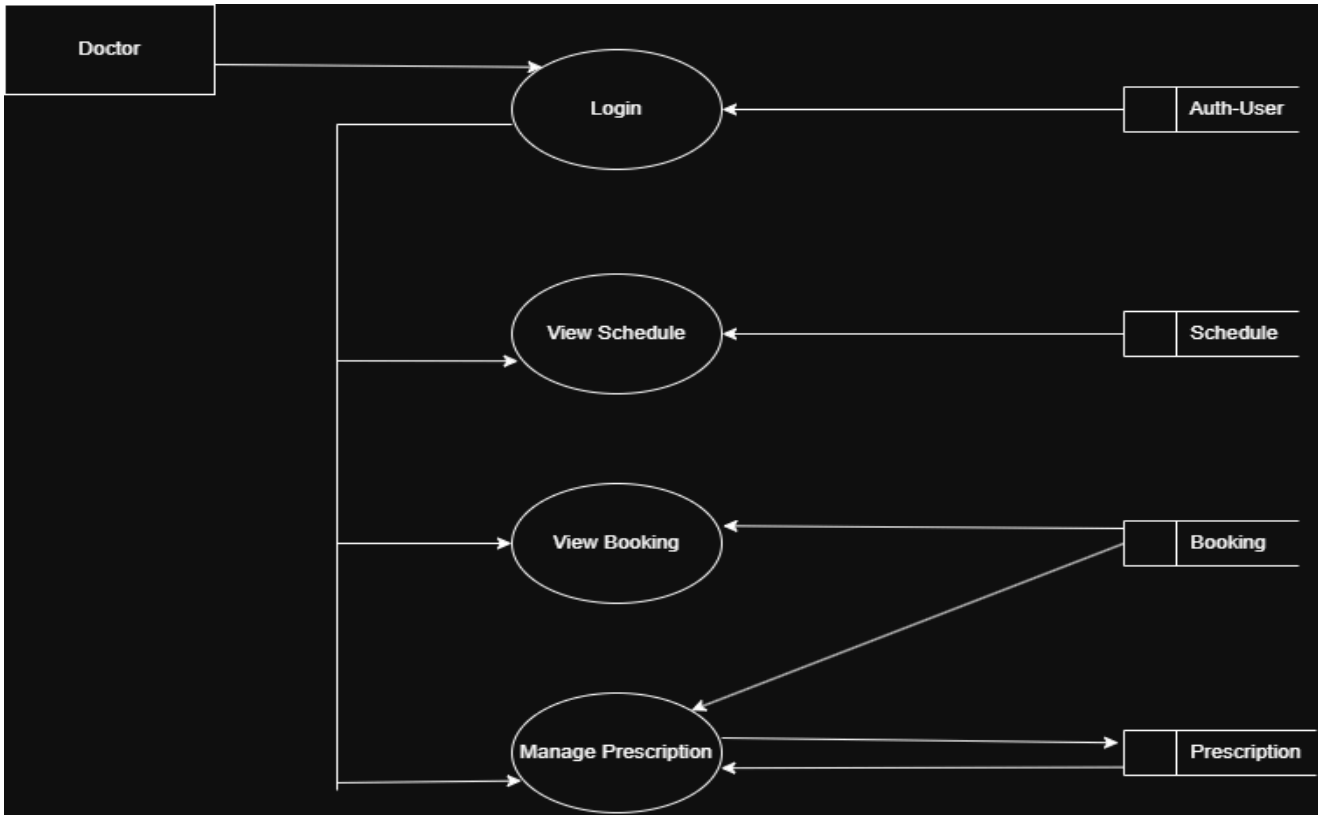
Level 0



Level 1



Level 2


Doctor
Login
Auth-User
View Schedule
Schedule
View Booking
Booking
Manage Prescription
Prescription

Level 3

2.4 Components Design

- Django Application
- Backup Module
- Restore Module
- Scheduler
- Cloud Integration

2.5 Key Design Considerations

- Reliability
- Redundancy
- Failover
- Scalability

- Security
- Performance

2.6 API Catalogue

- `run_full_backup()`
- `backup_database()`
- `backup_media()`
- `_upload_drive()`
- `run_restore()`
- `trigger_backup_async()`

---

3. Data Design

3.1 Data Model

Entities:

- Doctor
- User (Patient)
- Schedule
- Booking
- Prescription
- Complaint

Relationships:

- Doctor → Schedules
- User → Bookings
- Booking → Prescription
- User → Complaints

3.2 Data Access Mechanism

- CRUD using Django ORM
- Database backup using `mysqldump`
- Media handled via file system

3.3 Data Retention Policies

- Keep last **3 DB backups**
- Keep last **2 media backups**
- Old cloud backups deleted

3.4 Data Migration

- Backup → Store → Transfer → Restore
- Failover to cloud if local fails

---

4. Interfaces

4.1 User Interface

- Admin
- Doctor
- Patient

4.2 System Interface

- Django ↔ Database (ORM)
- Django ↔ Backup Module (Signals)

4.3 External Interfaces

- Google Drive API
- MySQL Database

4.4 File System Interface

- Stores `.sql` and `.zip` backups locally

4.5 Backup & Restore Interface

- Automatic backup trigger
- Command-based restore

---

5. State and Session Management

5.1 Session Management

- Django session framework
- Session ID via cookies
- Auto expiry and logout

5.2 Authentication

- Username/password login
- Role-based access:
    - Admin
    - Doctor
    - User

5.3 State Management

- Database state
- Backup state
- Application state

5.4 Backup Trigger State

- Triggered using `post_save` signal

5.5 Failover Handling

- Local backup priority
- Cloud fallback

---

6. Caching

6.1 Purpose

- Improve performance
- Reduce DB load

6.2 Types

- Application caching
- File system caching
- Session caching

6.3 Implementation

- Django caching (optional)
- Local backup as cache

6.4 Cache Expiry

- Updated when data changes
- Controlled via retention policy

6.5 Future Enhancements

- Redis / Memcached
- Distributed caching

---

7. Non-Functional Requirements

7.1 Security Aspects

- Secure login authentication
- OAuth for cloud access
- Protected database configuration
- Role-based access control

7.2 Performance Aspects

- Asynchronous backup execution
- Reduced response time using caching

- Efficient file handling

7.3 Reliability

- Multi-source backup
- Failover mechanism

7.4 Scalability

- Supports cloud expansion (AWS S3, Azure)

7.5 Availability

- Backup ensures data availability
- Restore mechanism ensures recovery

---

8. References

- Django Documentation
- Python Documentation
- Google Drive API Docs
- MySQL Documentation
- APScheduler Documentation